



# Hand Gesture Based Virtual Mouse

Web Application

---

B.Tech 3rd Year | IIOT / AI & ML Project | 2025–26

MediaPipe

OpenCV

Flask

Hugging Face



## Touchless Computer Control

Using Hand Gestures & AI

## What is this Project?

---

- 01 A web application that replaces the physical mouse with hand gestures
- 02 Uses a standard webcam — no special hardware required
- 03 Real-time hand landmark detection via Google's MediaPipe
- 04 Enables: cursor movement, click, scroll, and drag using fingers
- 05 Runs on Python Flask server, accessible from any browser
- 06 Deployed on Hugging Face Spaces for public access

# Problem Statement

*Why do we need a gesture-based mouse?*



## Accessibility Barriers

Physically disabled users struggle with traditional mice



## Hygiene & Touchless

Shared devices in public/medical environments pose infection risks



## Device Limitation

No mouse available? Touchpad limits precision and comfort



## HCI Modernization

Traditional input methods lack intuitiveness for AI-driven systems

# Objective

---

01

Develop a touchless, webcam-based mouse control system using hand gestures

02

Implement real-time hand tracking with MediaPipe for low-latency gesture detection

03

Map finger gestures to core mouse operations: move, click, scroll, and drag

04

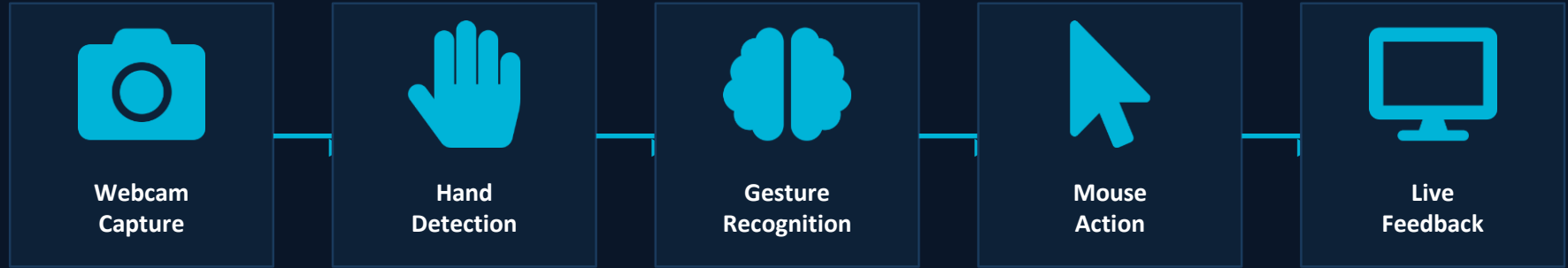
Build a web-based interface accessible via browser with no hardware dependency

05

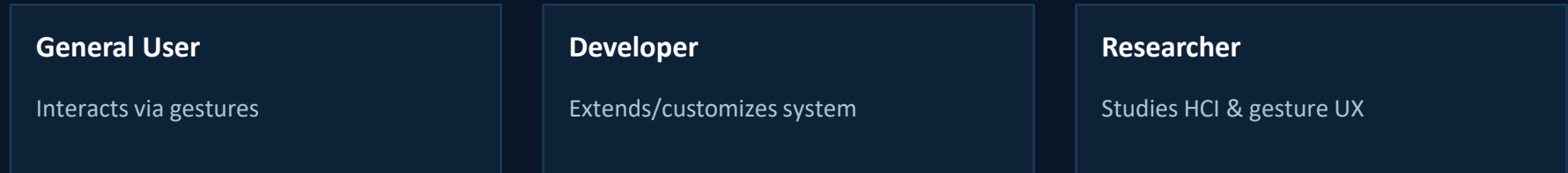
Deploy the application on Hugging Face Spaces for public, cloud-based accessibility

# System Overview

*End-to-end flow of the application*



## User Classes



# Technologies Used



## MediaPipe

*Hand landmark detection*

21 hand keypoints tracked in real-time at high FPS



## OpenCV

*Video capture & processing*

Handles webcam feed, frame manipulation & overlays



## Python Flask

*Backend web server*

Routes video stream, processes gestures, triggers mouse



## PyAutoGUI

*Mouse control library*

Executes actual OS-level mouse events from Python



## Hugging Face

*Cloud deployment platform*

Hosts the app on Spaces; free, public, scalable



## HTML/CSS/JS

*Frontend interface*

Browser UI for webcam access & real-time gesture feedback

# System Architecture

## FRONTEND

 HTML / CSS / JavaScript

 Webcam Access via `getUserMedia()`

 Video Streaming to Flask

 Gesture Feedback Display

  
HTTP/  
HTTPS

## BACKEND (Flask Server)

 Flask App (`app.py`)

 OpenCV — Webcam Capture

 MediaPipe — Hand Landmarks

 Gesture Logic — Finger States

 PyAutoGUI — Mouse Control

# Functional Requirements



FR1

## Live Video Capture

Capture real-time webcam feed via browser and stream to Flask server



FR2

## Hand Detection

Detect hand presence using MediaPipe Hands model with 21 landmarks



FR3

## Finger Tracking

Track individual finger positions and calculate movement vectors



FR4

## Gesture Recognition

Recognize gestures: index up (move), pinch (click), two-finger (scroll)



FR5

## Mouse Action Execution

Translate recognized gestures into OS mouse events via PyAutoGUI



# Non-Functional Requirements



## Performance

*Low latency*

Real-time processing at  $\geq 20$  FPS; minimal delay between gesture and action



## Reliability

*Continuous operation*

System should handle long sessions without crash or frame drops



## Security

*No video storage*

Webcam data processed in-memory; never stored or transmitted externally



## Usability

*Simple gestures*

Intuitive gestures requiring zero training for first-time users



## Scalability

*Multi-user support*

Cloud deployment on Hugging Face supports concurrent sessions



## Compatibility

*Cross-browser support*

Works on Chrome, Firefox, Edge; no plugin or extension needed

# Implementation Overview

## Backend

1. Create Flask app (app.py) with /stream route
2. Capture frames using cv2.VideoCapture()
3. Process each frame: BGR → RGB conversion
4. Run mp.solutions.hands for landmark detection
5. Map finger states to gesture codes
6. Execute pyautogui actions based on gesture

## Frontend

1. HTML page with <video> and <canvas> elements
2. Request webcam via getUserMedia() API
3. Stream video frames to Flask using fetch()
4. Display annotated frame with gesture labels
5. Show action status (Moving / Clicking / Scrolling)
6. Clean, minimal UI with connection status badge

# Deployment

## *Hugging Face Spaces*

1

### Prepare Files

Create requirements.txt, app.py, and a Gradio/Flask wrapper

2

### Push to GitHub

Commit all source files to a public GitHub repository

3

### Create HF Space

Go to [huggingface.co/spaces](https://huggingface.co/spaces) → New Space → Docker / SDK: Gradio

4

### Configure Runtime

Set Python version, install requirements, set gunicorn start command

5

### Deploy & Share

App goes live at [huggingface.co/spaces/your-username/app-name](https://huggingface.co/spaces/your-username/app-name)

## Why Hugging Face?

- ✓ Free tier for ML/AI projects
- ✓ Built-in support for Python & ML libs
- ✓ No credit card or billing required
- ✓ Public URL instantly accessible
- ✓ Git-based deploy — push to update
- ✓ Ideal for B.Tech project demos
- ✗ Render required paid plan for 24/7 uptime

# Future Enhancements

*What's next for this project?*



## Custom gesture training with personal ML models

Allow users to define and train their own gestures



## Multi-hand support for 2-hand interactions

Enable two-hand gestures for complex commands



## Mobile browser compatibility (iOS/Android)

Extend support to smartphone cameras and touch screens



## Virtual keyboard via air-typing gestures

Type without touching any physical keyboard



## Performance analytics dashboard

# Conclusion



Demonstrates a real-world AI application in Human-Computer Interaction



Fully functional with no special hardware — just a webcam and browser



Integrates leading tools: MediaPipe, OpenCV, Flask & PyAutoGUI



Deployed on Hugging Face Spaces — publicly accessible for free



Extensible and modular architecture ready for future upgrades

*"A step toward touchless, intuitive human-computer interaction"*

**Thank You!**